

SID4: 1066

PORT_BASE: 8166

PREFIX: s1066

SEED: 1066

VERIFY_SEED: 261066

DOMAIN_ID: 2

Hardware: Macintosh HD, Apple M4, 24 GB, Tahoe 26.6.2

Local Model Used: Ollama qwen3:8b

<https://github.com/paknathan/data260-1066/tree/main>

Part 1)

```

JS script.js > ...
1  const isValidSubmission = (description, Acceptance) => {
2      if (description.length <= 25) return "Description is too short (must be > 25 characters).";
3      const isChecked = document.getElementById(Acceptance).checked;
4      if (isChecked === false) return "You must accept the terms and conditions.";
5      return true;
6  };
7
8  function createCounter() {
9      let count = 0;
10     return function() {
11         count++;
12         return count;
13     };
14 }
15
16 const counter = createCounter();
17
18 document.getElementById("incidentForm").addEventListener("submit", (event) => {
19     event.preventDefault();
20
21     const submitButton = document.getElementById("submitButton");
22     const status = document.getElementById("status");
23     const errorMessage = document.getElementById("errorMessage");
24     const loadingIndicator = document.getElementById("loadingIndicator");
25
26     // Reset error container state
27     errorMessage.textContent = "";
28     errorMessage.classList.add("hidden");
29
30     const description = document.getElementById("description").value;
31     const result = isValidSubmission(description, "Acceptance");
32
33     // 1. ERROR STATE IMPLEMENTATION
34     if (result !== true) {
35         errorMessage.textContent = result;
36         errorMessage.classList.remove("hidden");
37         return;
38     }
39
40     // 2. LOADING STATE IMPLEMENTATION
41     submitButton.disabled = true;
42     submitButton.textContent = "Submitting...";
43     loadingIndicator.classList.add("hidden");
44     loadingIndicator.classList.remove("hidden");
45

```

```
45
46 // Simulate short network delay to display loading state
47 setTimeout(() => {
48     const formElement = document.querySelector("form");
49     const formData = new FormData(formElement);
50     const jsonString = JSON.stringify(Object.fromEntries(formData));
51     const jsonObject = JSON.parse(jsonString);
52
53     const { incident_id, email } = jsonObject;
54     const updatedJsonObject = { ...jsonObject, submissionDate: new Date() };
55
56     console.log("Logged JSON String:", jsonString);
57     console.log("Incident ID:", incident_id);
58     console.log("Email:", email);
59     console.log("Updated Object:", updatedJsonObject);
60     console.log("Submission Count:", counter());
61
62     // 3. CLEAR EMPTY STATE & DISPLAY SUCCESS
63     loadingIndicator.classList.add("hidden");
64     submitButton.disabled = false;
65     submitButton.textContent = "Submit";
66
67     status.textContent = `Incident ${incident_id} successfully logged to console!`;
68     status.classList.remove("hidden", "empty-state");
69     status.classList.add("success-state");
70
71     // Optional: Reset form fields after submission
72     formElement.reset();
73 }, 600);
74 });
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Municipal Transit Incidents</title>
  <style>
    /* Utility class to hide elements */
    .hidden {
      display: none !important;
    }

    /* Container constrained for 375px responsiveness */
    form {
      max-width: 375px;
      width: 100%;
      margin: 0 auto;
      padding: 1rem;
      display: flex;
      flex-direction: column;
      gap: 0.75rem;
      box-sizing: border-box;
    }

    input, textarea, select, button {
      width: 100%;
      box-sizing: border-box;
      padding: 0.5rem;
    }

    .state-message {
      padding: 0.5rem;
      border-radius: 4px;
      font-size: 0.875rem;
    }

    .empty-state {
      background-color: #f4f4f5;
      color: #52525b;
      border: 1px dashed #a1a1aa;
    }

    .error-state {
      background-color: #fef2f2;
      color: #991b1b;
      border: 1px solid #871717;
    }
  </style>

```

```

45     color: #991b1b;
46     border: 1px solid #f87171;
47   }
48 </style>
49 </head>
50 <body>
51
52   <form id="incidentForm">
53     <h1>Municipal transit incidents</h1>
54
55     <input type="text" id="incident_id" name="incident_id" required autofocus
56       placeholder="Enter the id of the incident (e.g., INC-00147)">
57     <input type="text" id="route_or_line" name="route_or_line" required
58       placeholder="Enter the affected route or line (e.g., Route 22, Blue Line)">
59     <input type="text" id="email" name="email" required placeholder="Enter your email">
60     <textarea id="description" name="description" required
61       placeholder="Describe what happened (e.g., cause, impact on riders, actions taken)"></textarea>
62
63     <select name="Incident Type" id="Incident Type">
64       <option value="Mechanical Breakdown">Mechanical Breakdown</option>
65       <option value="Collision">Collision</option>
66       <option value="Delay">Delay</option>
67       <option value="Route Detour">Route Detour</option>
68     </select>
69
70     <div class="checkbox-container">
71       <input type="checkbox" id="Acceptance" name="Terms & Conditions" value="yes">
72       <label for="Acceptance">I agree to the terms and conditions.</label>
73     </div>
74
75     <!-- Error State Container -->
76     <div id="errorMessage" class="state-message error-state hidden" role="alert"></div>
77
78     <!-- Loading Indicator -->
79     <div id="loadingIndicator" class="hidden">
80       Processing incident submit...
81     </div>
82
83     <button type="submit" id="submitButton">Submit</button>
84
85     <!-- Empty/Status Container -->
86     <div id="status" class="state-message empty-state">No incidents submitted yet. Fill out the form to log a record.</div>
87   </form>

```

```

84
85     <!-- Empty/Status Container -->
86     <div id="status" class="state-message empty-state">No incidents submitted yet. Fill out the form to log a record.</div>
87   </form>
88
89   <script src="script.js"></script>
90 </body>
91 </html>

```

```
# style.css > ...
1  /* Responsive container up to 375px mobile breakpoint */
2  form {
3      max-width: 375px;
4      width: 100%;
5      margin: 0 auto;
6      padding: 1rem;
7      box-sizing: border-box;
8      display: flex;
9      flex-direction: column;
10     gap: 0.75rem;
11 }
12
13 input, textarea, select, button {
14     width: 100%;
15     box-sizing: border-box;
16     padding: 0.5rem;
17 }
18
19 .hidden {
20     display: none !important;
21 }
22
23 /* Base message state styling */
24 .state-message {
25     padding: 0.75rem;
26     border-radius: 4px;
27     font-size: 0.9rem;
28 }
29
30 /* Empty State */
31 .empty-state {
32     background-color: #f4f4f5;
33     color: #71717a;
34     border: 1px dashed #d4d4d8;
35 }
36
37 /* Error State */
38 .error-state {
39     background-color: #fef2f2;
40     color: #991b1b;
41     border: 1px solid #f87171;
42 }
43
```

```

43
44  /* Success State */
45  .success-state {
46      background-color: #f0fdf4;
47      color: #166534;
48      border: 1px solid #4ade80;
49  }
50
51  /* Loading Spinner */
52  .loading-state {
53      background-color: #fff6ff;
54      color: #1e40af;
55      border: 1px solid #60a5fa;
56      display: flex;
57      align-items: center;
58      gap: 0.5rem;
59  }
60
61  .spinner {
62      width: 14px;
63      height: 14px;
64      border: 2px solid #93c5fd;
65      border-top-color: #1d4ed8;
66      border-radius: 50%;
67      animation: spin 0.8s linear infinite;
68  }
69
70  @keyframes spin {
71      to { transform: rotate(360deg); }
72  }

```

Empty State:

Municipal transit incidents

Enter the id of the incident (e.g., INC-00147)

Enter the affected route or line (e.g., Route 22, Blue Lir

Enter your email

Describe what happened (e.g., cause, impact on riders, actions taken)

Mechanical Breakdown

☐

I agree to the terms and conditions.

Submit

No incidents submitted yet. Fill out the form to log a record.

Error State:

Municipal transit incidents

67455

CA-85 N

nathan.pak@sjsu.edu

I got rear ended.

Collision

☒

I agree to the terms and conditions.

Description is too short (must be > 25 characters).

Submit

No incidents submitted yet. Fill out the form to log a record.

Loading State:

The screenshot shows a web browser window with the address bar displaying 'localhost:8000'. The page title is 'Municipal transit incidents'. The form contains the following elements:

- Input field with value '67455'
- Input field with value 'CA-85 N'
- Input field with value 'nathan.pak@sjsu.edu'
- Text area with value 'I got rear ended during morning congestion.'
- Dropdown menu with value 'Collision'
- Checkbox with value 'I agree to the terms and conditions.'
- Text 'Processing incident submit...'
- Submit button with text 'Submitting...'
- Message box with text 'No incidents submitted yet. Fill out the form to log a record.'

Part 2: FastAPI

1. Add a FastAPI backend on your PORT_BASE. Write the code to add a new record for your domain entity. The user should be able to enter values for the primary and secondary fields (e.g., if your domain entity is a restaurant, the primary field might be the restaurant name and the secondary field Cuisine). Once the user submits the required data, the record should be added, and the user should be redirected to the home view showing the updated list

```
main.py > ...
1  from fastapi import FastAPI, Form, Request
2  from fastapi.responses import HTMLResponse, RedirectResponse
3  from fastapi.templating import Jinja2Templates
4  import uvicorn
5
6  app = FastAPI(title="Municipal Transit Incident API")
7
8  templates = Jinja2Templates(directory="templates")
9
10 incidents_db = []
11
12 @app.get("/", response_class=HTMLResponse)
13 def read_root(request: Request):
14     return templates.TemplateResponse(
15         "index.html",
16         {"request": request, "incidents": incidents_db}
17     )
18
19 @app.post("/add")
20 def add_incident(
21     incident_id: str = Form(...),
22     route_or_line: str = Form(...),
23     incident_type: str = Form(...),
24     email: str = Form(...),
25     description: str = Form(...)
26 ):
27     new_record = {
28         "incident_id": incident_id,
29         "route_or_line": route_or_line,
30         "incident_type": incident_type,
31         "email": email,
32         "description": description
33     }
34     incidents_db.append(new_record)
35
36     # 303 Redirect back to GET home view
37     return RedirectResponse(url="/", status_code=303)
38
39 if __name__ == "__main__":
40     uvicorn.run("main:app", host="0.0.0.0", port=8166, reload=True)
```

JS script.js > [⌘] isValidSubmission

```
1  ✓ const isValidSubmission = (description, Acceptance) => {  
2    if (description.length <= 25) return "Description is too short (must be > 25 characters).";  
3    const isChecked = document.getElementById(Acceptance).checked;  
4    if (isChecked === false) return "You must accept the terms and conditions.";  
5    return true;  
6  };  
7  
8  ✓ function createCounter() {  
9    let count = 0;  
10   ✓ return function() {  
11     count++;  
12     return count;  
13   };  
14 }  
15  
16 const counter = createCounter();  
17  
18 ✓ document.getElementById("incidentForm").addEventListener("submit", (event) => {  
19   event.preventDefault();  
20  
21   const submitButton = document.getElementById("submitButton");  
22   const status = document.getElementById("status");  
23   const errorMessage = document.getElementById("errorMessage");  
24   const loadingIndicator = document.getElementById("loadingIndicator");  
25  
26   // Reset error container state  
27   errorMessage.textContent = "";  
28   errorMessage.classList.add("hidden");  
29  
30   const description = document.getElementById("description").value;  
31   const result = isValidSubmission(description, "Acceptance");  
32  
33   // 1. ERROR STATE IMPLEMENTATION  
34   ✓ if (result !== true) {  
35     errorMessage.textContent = result;  
36     errorMessage.classList.remove("hidden");  
37     return;  
38   }  
39  
40   // 2. LOADING STATE IMPLEMENTATION  
41   submitButton.disabled = true;  
42   submitButton.textContent = "Submitting...";  
43   loadingIndicator.classList.add("hidden");  
44   loadingIndicator.classList.remove("hidden");  
45  
46   // Simulate short network delay to display loading state  
47   ✓ setTimeout(() => {
```

```

45
46 // Simulate short network delay to display loading state
47 setTimeout(() => {
48     const formElement = document.querySelector("form");
49     const formData = new FormData(formElement);
50     const jsonString = JSON.stringify(Object.fromEntries(formData));
51     const jsonObject = JSON.parse(jsonString);
52
53     const { incident_id, email } = jsonObject;
54     const updatedJsonObject = { ...jsonObject, submissionDate: new Date() };
55
56     console.log("Logged JSON String:", jsonString);
57     console.log("Incident ID:", incident_id);
58     console.log("Email:", email);
59     console.log("Updated Object:", updatedJsonObject);
60     console.log("Submission Count:", counter());
61
62 // 3. CLEAR EMPTY STATE & DISPLAY SUCCESS
63 loadingIndicator.classList.add("hidden");
64 submitButton.disabled = false;
65 submitButton.textContent = "Submit";
66
67 status.textContent = `Incident ${incident_id} successfully logged to console!`;
68 status.classList.remove("hidden", "empty-state");
69 status.classList.add("success-state");
70
71 // Optional: Reset form fields after submission
72 formElement.reset();
73 }, 600);
74 });

```

```
templates > index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Municipal Transit Incidents</title>
7   <style>
8     body { font-family: Arial, sans-serif; margin: 2rem; max-width: 600px; }
9     form { display: flex; flex-direction: column; gap: 0.75rem; margin-bottom: 2rem; }
10    input, textarea, select, button { padding: 0.5rem; width: 100%; box-sizing: border-box; }
11    table { width: 100%; border-collapse: collapse; margin-top: 1rem; }
12    th, td { border: 1px solid #ccc; padding: 0.5rem; text-align: left; }
13    th { background-color: #f4f4f5; }
14  </style>
15 </head>
16 <body>
17
18   <h1>Municipal transit incidents</h1>
19
20   <!-- Ensure action and method point to FastAPI -->
21   <form action="/add" method="POST" id="incidentForm">
22     <input type="text" id="incident_id" name="incident_id" required autofocus
23       placeholder="Enter the id of the incident (e.g., INC-00147)">
24
25     <input type="text" id="route_or_line" name="route_or_line" required
26       placeholder="Enter the affected route or line (e.g., Route 22, Blue Line)">
27
28     <input type="text" id="email" name="email" required placeholder="Enter your email">
29
30     <textarea id="description" name="description" required
31       placeholder="Describe what happened (e.g., cause, impact on riders, actions taken)"></textarea>
32
33     <select name="incident_type" id="incident_type">
34       <option value="Mechanical Breakdown">Mechanical Breakdown</option>
35       <option value="Collision">Collision</option>
36       <option value="Delay">Delay</option>
37       <option value="Route Detour">Route Detour</option>
38     </select>
39
40     <div>
41       <input type="checkbox" id="Acceptance" name="Acceptance" value="yes" required>
42       <label for="Acceptance">I agree to the terms and conditions.</label>
43     </div>
44
45     <button type="submit" id="submitButton">Submit</button>
46   </form>
47
```

```

46     {% endif %}
47
48     <hr>
49
50     <!-- Updated List View Section -->
51     <h2>Logged Incidents</h2>
52     {% if incidents %}
53     <table>
54         <thead>
55             <tr>
56                 <th>Incident ID</th>
57                 <th>Route / Line</th>
58                 <th>Type</th>
59                 <th>Email</th>
60                 <th>Description</th>
61             </tr>
62         </thead>
63         <tbody>
64             {% for incident in incidents %}
65             <tr>
66                 <td>{{ incident.incident_id }}</td>
67                 <td>{{ incident.route_or_line }}</td>
68                 <td>{{ incident.incident_type }}</td>
69                 <td>{{ incident.email }}</td>
70                 <td>{{ incident.description }}</td>
71             </tr>
72             {% endfor %}
73         </tbody>
74     </table>
75     {% else %}
76     <p>No transit incidents submitted yet.</p>
77     {% endif %}
78
79 </body>
80 </html>

```

Municipal transit incidents

Enter the id of the incident (e.g., INC-00147)

Enter the affected route or line (e.g., Route 22, Blue Line)

Enter your email

Describe what happened (e.g., cause, impact on riders, actions taken)

Mechanical Breakdown

☐

I agree to the terms and conditions.

Submit

Logged Incidents

Incident ID	Route / Line	Type	Email	Description
67455	CA-85 N	Collision	nathan.pak@sjsu.edu	I got rear ended during morning congestion.

2) Write the code to update the record with ID 1 to domain-appropriate values for the primary and secondary fields. After submitting the data, redirect to the home view and show the updated data in the list.

templates > <> index.html > ...

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Municipal Transit Incidents</title>
7    <style>
8      body { font-family: Arial, sans-serif; margin: 2rem; max-width: 600px; }
9      form { display: flex; flex-direction: column; gap: 0.75rem; margin-bottom: 2rem; }
10     input, textarea, select, button { padding: 0.5rem; width: 100%; box-sizing: border-box; }
11     table { width: 100%; border-collapse: collapse; margin-top: 1rem; }
12     th, td { border: 1px solid #ccc; padding: 0.5rem; text-align: left; }
13     th { background-color: #f4f4f5; }
14
15     .update-banner {
16       background-color: #e6f7ec;
17       border: 1px solid #34a853;
18       color: #1e7a3a;
19       padding: 0.75rem 1rem;
20       border-radius: 4px;
21       margin-top: 1rem;
22       font-weight: bold;
23     }
24
25     @keyframes flash-highlight {
26       0% { background-color: #c8f2d4; }
27       100% { background-color: transparent; }
28     }
29     .row-updated {
30       animation: flash-highlight 2s ease-out;
31     }
32   </style>
33 </head>
34 <body>
35
36   <h1>Municipal transit incidents</h1>
37
38   <!-- Ensure action and method point to FastAPI -->
39   <form action="/add" method="POST" id="incidentForm">
40     <input type="text" id="incident_id" name="incident_id" required autofocus
41       placeholder="Enter the id of the incident (e.g., INC-00147)">
42
43     <input type="text" id="route_or_line" name="route_or_line" required
44       placeholder="Enter the affected route or line (e.g., Route 22, Blue Line)">
45
46     <input type="text" id="email" name="email" required placeholder="Enter your email">
47
```

```

46 <input type="text" id="email" name="email" required placeholder="Enter your email">
47
48 <textarea id="description" name="description" required
49 | placeholder="Describe what happened (e.g., cause, impact on riders, actions taken)"></textarea>
50
51 <select name="incident_type" id="incident_type">
52 | <option value="Mechanical Breakdown">Mechanical Breakdown</option>
53 | <option value="Collision">Collision</option>
54 | <option value="Delay">Delay</option>
55 | <option value="Route Detour">Route Detour</option>
56 </select>
57
58 <div>
59 | <input type="checkbox" id="Acceptance" name="Acceptance" value="yes" required>
60 | <label for="Acceptance">I agree to the terms and conditions.</label>
61 </div>
62
63 <button type="submit" id="submitButton">Submit</button>
64 </form>
65
66 <hr>
67
68 <!-- Updated List View Section -->
69 <h2>Logged Incidents</h2>
70
71 {% if updated_id %}
72 <div class="update-banner">
73 | &#10003; Incident {{ updated_id }} updated successfully.
74 </div>
75 {% endif %}
76
77 {% if incidents %}
78 <table>
79 | <thead>
80 | | <tr>
81 | | | <th>Incident ID</th>
82 | | | <th>Route / Line</th>
83 | | | <th>Type</th>
84 | | | <th>Email</th>
85 | | | <th>Description</th>
86 | | | <th></th>
87 | | </tr>
88 | </thead>
89 | <tbody>

```

```

89     <tbody>
90         {% for incident in incidents %}
91         <tr {% if incident.incident_id == updated_id %}class="row-updated"{% endif %}>
92             <td>{{ incident.incident_id }}</td>
93             <td>
94                 <input type="text" name="route_or_line" value="{{ incident.route_or_line }}"
95                     form="update-form-{{ loop.index }}" required>
96             </td>
97             <td>
98                 <select name="incident_type" form="update-form-{{ loop.index }}">
99                     {% for option in ["Mechanical Breakdown", "Collision", "Delay", "Route Detour"] %}
100                     <option value="{{ option }}" {% if incident.incident_type == option %}selected{% endif %}>{{ option }}</option>
101                     {% endfor %}
102                 </select>
103             </td>
104             <td>{{ incident.email }}</td>
105             <td>{{ incident.description }}</td>
106             <td>
107                 <!-- Empty form: holds the action/method that the inputs above bind to via form="..." -->
108                 <form id="update-form-{{ loop.index }}" action="/update/{{ incident.incident_id }}" method="POST"></form>
109                 <button type="submit" form="update-form-{{ loop.index }}">Update</button>
110             </td>
111         </tr>
112         {% endfor %}
113     </tbody>
114 </table>
115 {% else %}
116 <p>No transit incidents submitted yet.</p>
117 {% endif %}
118
119 </body>
120 </html>

```

```

JS script.js >  addEventListeners("submit") callback >  setTimeout() callback
1  const isValidSubmission = (description, Acceptance) => {
2      if (description.length <= 25) return "Description is too short (must be > 25 characters).";
3      const isChecked = document.getElementById(Acceptance).checked;
4      if (isChecked === false) return "You must accept the terms and conditions.";
5      return true;
6  };
7
8  function createCounter() {
9      let count = 0;
10     return function() {
11         count++;
12         return count;
13     };
14 }
15
16 const counter = createCounter();
17
18 document.getElementById("incidentForm").addEventListener("submit", (event) => {
19     event.preventDefault();
20
21     const submitButton = document.getElementById("submitButton");
22     const status = document.getElementById("status");
23     const errorMessage = document.getElementById("errorMessage");
24     const loadingIndicator = document.getElementById("loadingIndicator");
25
26     // Reset error container state
27     errorMessage.textContent = "";
28     errorMessage.classList.add("hidden");
29
30     const description = document.getElementById("description").value;
31     const result = isValidSubmission(description, "Acceptance");
32
33     // 1. ERROR STATE IMPLEMENTATION
34     if (result !== true) {
35         errorMessage.textContent = result;
36         errorMessage.classList.remove("hidden");
37         return;
38     }
39
40     // 2. LOADING STATE IMPLEMENTATION
41     submitButton.disabled = true;
42     submitButton.textContent = "Submitting...";
43     loadingIndicator.classList.add("hidden");
44     loadingIndicator.classList.remove("hidden");
45

```

```

45
46 // Simulate short network delay to display loading state
47 setTimeout(() => {
48     const formElement = document.querySelector("form");
49     const formData = new FormData(formElement);
50     const jsonString = JSON.stringify(Object.fromEntries(formData));
51     const jsonObject = JSON.parse(jsonString);
52
53     const { incident_id, email } = jsonObject;
54     const updatedJsonObject = { ...jsonObject, submissionDate: new Date() };
55
56     console.log("Logged JSON String:", jsonString);
57     console.log("Incident ID:", incident_id);
58     console.log("Email:", email);
59     console.log("Updated Object:", updatedJsonObject);
60     console.log("Submission Count:", counter());
61
62     // 3. CLEAR EMPTY STATE & DISPLAY SUCCESS
63     loadingIndicator.classList.add("hidden");
64     submitButton.disabled = false;
65     submitButton.textContent = "Submit";
66
67     status.textContent = `Incident ${incident_id} successfully logged to console!`;
68     status.classList.remove("hidden", "empty-state");
69     status.classList.add("success-state");
70
71     // Optional: Reset form fields after submission
72     formElement.reset();
73 }, 600);
74 });

```

Municipal transit incidents

Enter the id of the incident (e.g., INC-00147)

Enter the affected route or line (e.g., Route 22, Blue Line)

Enter your email

Describe what happened (e.g., cause, impact on riders, actions taken)

Mechanical Breakdown

☐

I agree to the terms and conditions.

Submit

Logged Incidents

✓ Incident 67455 updated successfully.

Incident ID	Route / Line	Type	Email	Description	
INC-00147	US-10	M	operator@transit.org	Engine failure near 4th street stop.	Update
67455	CA-85	D	nathan.pak@sjsu.edu	I got rear ended during morning congestion.	Update

3) Write the code to delete the record with the highest ID. After submitting the data, redirect to the home view and show the updated data in the list.

```

67
68     <!-- Updated List View Section -->
69     <h2>Logged Incidents</h2>
70
71     {% if incidents %}
72     <form action="/delete-highest" method="POST" style="margin-bottom: 1rem;">
73         <button type="submit" style="width: auto; background-color: #fdecea; border: 1px solid #d93025; color: #a50e0e;">
74             Delete Highest ID
75         </button>
76     </form>
77     {% endif %}
78

```

```

48 @app.post("/update/{incident_id}")
49 def update_incident(
50     incident_id: str,
51     route_or_line: str = Form(...),
52     incident_type: str = Form(...)
53 ):
54     # Update the primary (route/line) and secondary (incident type) fields
55     # for the matching record with whatever values were submitted.
56     for incident in incidents_db:
57         if incident["incident_id"] == incident_id:
58             incident["route_or_line"] = route_or_line
59             incident["incident_type"] = incident_type
60             break
61
62     # Redirect back to GET home view, flagging which record just changed
63     return RedirectResponse(url=f"/?updated={incident_id}", status_code=303)
64
65 def _numeric_id(incident_id: str) -> int:
66     """Extract the numeric portion of an incident_id like 'INC-00147' -> 147.
67     Records with no digits at all sort lowest (treated as -1)."""
68     digits = re.findall(r"\d+", incident_id)
69     return int("".join(digits)) if digits else -1
70
71 @app.post("/delete-highest")
72 def delete_highest_incident():
73     if incidents_db:
74         highest = max(incidents_db, key=lambda incident: _numeric_id(incident["incident_id"]))
75         incidents_db.remove(highest)
76
77     # Redirect back to GET home view, which re-renders the (now shorter) list
78     return RedirectResponse(url="/", status_code=303)
79
80 if __name__ == "__main__":
81     uvicorn.run("main:app", host="0.0.0.0", port=8166, reload=True)

```

Municipal transit incidents

Mechanical Breakdown

▼

☐

I agree to the terms and conditions.

Submit

Logged Incidents

Delete Highest ID

Incident ID	Route / Line	Type	Email	Description	
67455	CA-85	Ct ▼	nathan.pak@sjsu.edu	I got rear ended during morning congestion.	Update

4. Write the code to add search functionality. The user should be able to search for a record by the primary or secondary field. When the user enters a value and searches, the list should update to show only the matching records.

```

<hr>

<!-- Updated List View Section -->
<h2>Logged Incidents</h2>

<!-- Search by primary (route/line) or secondary (incident type) field -->
<form action="/" method="GET" class="search-bar">
  <input type="text" name="q" value="{{ search_query }}"
    | placeholder="Search by route/line or incident type..."
  <button type="submit">Search</button>
  {% if search_query %}
  <a href="/" class="clear-link">Clear</a>
  {% endif %}
</form>

{% if has_incidents %}
<form action="/delete-highest" method="POST" style="margin-bottom: 1rem;">
  <button type="submit" style="width: auto; background-color: #fdecea; border: 1px solid #d93025; color: #a50e0e;">
    Delete Highest ID
  </button>
</form>
{% endif %}

```

```

main.py > add_incident
15     incident_type = "Mechanical Breakdown",
16     "email": "operator@transit.org",
17     "description": "Engine failure near 4th street stop."
18 }
19 ]
20
21 @app.get("/", response_class=HTMLResponse)
22 def read_root(request: Request, updated: str | None = None, q: str | None = None):
23     search_query = (q or "").strip()
24
25     if search_query:
26         needle = search_query.lower()
27         filtered_incidents = [
28             incident for incident in incidents_db
29             if needle in incident["route_or_line"].lower()
30             or needle in incident["incident_type"].lower()
31         ]
32     else:
33         filtered_incidents = incidents_db
34
35     return templates.TemplateResponse(
36         "index.html",
37         {
38             "request": request,
39             "incidents": filtered_incidents,
40             "updated_id": updated,
41             "search_query": search_query,
42             "has_incidents": bool(incidents_db),
43         }
44     )
45
46 @app.post("/add")
47 def add_incident(
48     incident_id: str = Form(...),
49     route_or_line: str = Form(...),
50     incident_type: str = Form(...),
51     email: str = Form(...),
52     description: str = Form(...)
53 ):
54     new_record = {
55         "incident_id": incident_id,
56         "route_or_line": route_or_line,
57         "incident_type": incident_type,
58         "email": email,
59         "description": description
60     }
61     incidents_db.append(new_record)
62

```

Municipal transit incidents

Mechanical Breakdown

▼

☐

I agree to the terms and conditions.

Submit

Logged Incidents

Search

Clear

Delete Highest ID

Incident ID	Route / Line	Type	Email	Description	
67455	CA-8	(▼	nathan.pak@sjsu.edu	I got rear ended during morning congestion.	Update
3033822523	CA-8	(▼	nathan.pak@sjsu.edu	Two cars crashed into each other head on.	Update

```

agent_graph.py > ...
1  import argparse
2  import json
3  import sys
4  from typing import Any, Dict, List, Optional, TypedDict
5
6  from langgraph.graph import StateGraph, END
7  from pydantic import BaseModel, Field, field_validator, ValidationError
8
9  from src.model_client import complete, ModelClient
10
11 # -----
12 # Config (Part 0 values would normally be pulled in from your SID4 config
13 # module -- import them here instead of hardcoding once that module exists)
14 # -----
15 DEFAULT_MODEL = "qwen3:8b"
16 DEFAULT_TASK = "tag_and_summarize"
17 DEFAULT_TURN_CEILING = 6 # Part 4 compares ceilings of 2 and 10 explicitly;
18 | | | | | | | | | | # this is just the fallback if state doesn't set one.
19 NUM_TAGS = 3
20 TAG_MIN_CHARS = 3
21 TAG_MAX_CHARS = 30
22 MAX_SUMMARY_WORDS = 25
23
24
25 # -----
26 # Step 2: Shared state
27 # -----
28 class AgentState(TypedDict, total=False):
29     # Inputs
30     title: str # e.g. "Route 22 -- Mechanical Breakdown"
31     content: str # free-text incident description / details
32     email: str # pass-through; not acted on by this graph yet
33     strict: bool # True = hard-fail-and-retry on schema violations;
34     | | | | | | | | | | # False = attempt a local auto-fix first
35     task: str # which prompt template to run
36     llm: Any # optional model name override (str) or
37     | | | | | | | | | | # a pre-built ModelClient instance
38
39     # Working memory
40     planner_proposal: Dict[str, Any] # {"candidate_tags": [...], "draft_summary": "..."}
41     reviewer_feedback: Dict[str, Any] # {"valid": bool, "tags":..., "summary":..., "error":...}
42
43     # Loop control
44     turn_count: int
45     turn_ceiling: int
46
47

```

```

47
48 # -----
49 # Part 4 (1): Pydantic schema for the Reviewer's final output
50 # -----
51 class ReviewedOutput(BaseModel):
52     tags: List[str] = Field(..., min_length=NUM_TAGS, max_length=NUM_TAGS)
53     summary: str
54
55     @field_validator("tags")
56     @classmethod
57     def check_tag_shape(cls, v: List[str]) -> List[str]:
58         for t in v:
59             if not isinstance(t, str) or not (TAG_MIN_CHARS <= len(t.strip()) <= TAG_MAX_CHARS):
60                 raise ValueError(
61                     f"tag {t!r} must be a string of {TAG_MIN_CHARS}-{TAG_MAX_CHARS} characters"
62                 )
63         return v
64
65     @field_validator("summary")
66     @classmethod
67     def check_summary_length(cls, v: str) -> str:
68         word_count = len((v or "").split())
69         if word_count == 0 or word_count > MAX_SUMMARY_WORDS:
70             raise ValueError(
71                 f"summary must be 1-{MAX_SUMMARY_WORDS} words, got {word_count}"
72             )
73         return v
74
75 # -----
76 # Model adapter call -- routed through src/model_client.py only, per spec.
77 # Honors state["llm"] as an optional per-run model override.
78 # -----
79
80 def call_model(state: AgentState, system_prompt: str, user_prompt: str) -> dict:
81     messages = [
82         {"role": "system", "content": system_prompt},
83         {"role": "user", "content": user_prompt},
84     ]
85     llm_override = state.get("llm")
86     if isinstance(llm_override, ModelClient):
87         return llm_override.complete(messages=messages, response_format="json")
88     if isinstance(llm_override, str) and llm_override:
89         return ModelClient(model=llm_override).complete(messages=messages, response_format="json")
90     return complete(messages=messages, response_format="json")
91
92

```

```

92
93 # -----
94 # Prompt templates, keyed by state["task"]. Only one task exists today;
95 # this makes it straightforward to add more later without touching the
96 # node functions themselves.
97 # -----
98 def _planner_prompts(state: AgentState):
99     system_prompt = (
100         "You are the Planner agent in a multi-agent pipeline that tags and "
101         "summarizes Municipal Transit Incident records (bus/subway/light "
102         "rail/streetcar/commuter rail/ferry disruptions). Given a title and "
103         "content describing one incident, propose 5 to 7 candidate topical "
104         "tags (short lowercase phrases) grounded in specific details from "
105         "the content -- e.g. the incident type, transit mode, route/line, "
106         "location, or severity if mentioned -- and a rough draft summary "
107         "(1-3 sentences, will be trimmed later). Do not invent details that "
108         "aren't in the title/content. "
109         "Respond ONLY with JSON in this exact shape: "
110         '{"candidate_tags": [...], "...", ...], "draft_summary": "..."}'
111     )
112     user_prompt = f"Title: {state['title']}\nContent: {state['content']}"
113
114     feedback = state.get("reviewer_feedback") or {}
115     if feedback.get("valid") is False and feedback.get("error"):
116         user_prompt += (
117             f"\n\nYour previous draft was rejected. Validation error: "
118             f'"{feedback["error"]}"\nProduce a corrected draft that fixes this "'
119             f'exactly (tags must be {TAG_MIN_CHARS}-{TAG_MAX_CHARS} characters "'
120             f'each, and there must be room to trim the summary to "'
121             f'{MAX_SUMMARY_WORDS} words or fewer)."'
122         )
123     return system_prompt, user_prompt
124
125
126 def _reviewer_prompts(state: AgentState):
127     system_prompt = (
128         "You are the Reviewer agent in a multi-agent pipeline for Municipal "
129         f"Transit Incident records. You will receive a title, content, and "
130         f"a Planner's draft (candidate tags + draft summary). Select "
131         f"exactly {NUM_TAGS} of the most topical tags from the candidates "
132         f"(each must be {TAG_MIN_CHARS}-{TAG_MAX_CHARS} characters), "
133         "keeping only ones clearly grounded in the content. Rewrite the "
134         f"summary to be at most {MAX_SUMMARY_WORDS} words, factual and "
135         "specific -- do not add information that isn't in the title/content. "
136         "Respond ONLY with JSON in this exact shape: "
137         '{"tags": [...], "...", "..."], "summary": "..."}'
138     )
139     proposal = state.get("planner_proposal") or {}
140     user_prompt = (
141         f"Title: {state['title']}\nContent: {state['content']}\n\n"
142         f"Planner draft: {json.dumps(proposal)}"
143     )

```

```

144     return system_prompt, user_prompt
145
146
147 # -----
148 # Step 3: Agent nodes
149 # -----
150 def planner_node(state: AgentState) -> Dict[str, Any]:
151     print("---NODE: Planner ---", file=sys.stderr)
152     system_prompt, user_prompt = _planner_prompts(state)
153     out = call_model(state, system_prompt, user_prompt)
154     proposal = {
155         "candidate_tags": out.get("candidate_tags", []),
156         "draft_summary": out.get("draft_summary", ""),
157     }
158     # Clear any prior rejection now that a fresh proposal exists.
159     return {"planner_proposal": proposal, "reviewer_feedback": {}}
160
161
162 def reviewer_node(state: AgentState) -> Dict[str, Any]:
163     print("---NODE: Reviewer ---", file=sys.stderr)
164     system_prompt, user_prompt = _reviewer_prompts(state)
165     out = call_model(state, system_prompt, user_prompt)
166     tags = out.get("tags", [])
167     summary = out.get("summary", "")
168
169     try:
170         validated = ReviewedOutput(tags=tags, summary=summary)
171         return {
172             "reviewer_feedback": {
173                 "valid": True,
174                 "tags": validated.tags,
175                 "summary": validated.summary,
176             }
177         }
178     except ValidationError as exc:
179         if not state.get("strict", True):
180             # Lenient mode: try a cheap local repair before giving up on it.
181             fixed_tags = [t.strip() for t in tags if isinstance(t, str) and t.strip()]
182             fixed_tags = (fixed_tags + ["general incident", "transit delay", "service issue"][:NUM_TAGS])
183             fixed_tags = [t[:TAG_MAX_CHARS] for t in fixed_tags]
184             words = (summary or "").split()
185             fixed_summary = " ".join(words[:MAX_SUMMARY_WORDS]) or "Incident summary unavailable."
186             try:
187                 validated = ReviewedOutput(tags=fixed_tags, summary=fixed_summary)
188                 return {
189                     "reviewer_feedback": {
190                         "valid": True,
191                         "tags": validated.tags,
192                         "summary": validated.summary,
193                     }
194                 }
195             except ValidationError:
196                 # If the lenient repair also fails, fall back to the original error.
197                 raise exc

```

```

189         reviewer_feedback: {
190             "valid": True,
191             "tags": validated.tags,
192             "summary": validated.summary,
193             "auto_repaired": True,
194         }
195     }
196 except ValidationError:
197     pass # fall through to reporting the original failure
198
199     return {"reviewer_feedback": {"valid": False, "error": str(exc)}}
200
201
202 def supervisor_node(state: AgentState) -> Dict[str, Any]:
203     """State-updating half of the Supervisor: just advances the turn
204     counter. All branching decisions live in router_logic below."""
205     turn_count = state.get("turn_count", 0) + 1
206     print(f"---NODE: Supervisor (turn {turn_count}) ---", file=sys.stderr)
207     return {"turn_count": turn_count}
208
209
210 # -----
211 # Step 4: Routing function -- reads state, returns "planner", "reviewer", or END
212 # -----
213 def router_logic(state: AgentState) -> str:
214     turn_count = state.get("turn_count", 0)
215     ceiling = state.get("turn_ceiling", DEFAULT_TURN_CEILING)
216
217     if turn_count >= ceiling:
218         print(f" [Router] turn ceiling ({ceiling}) reached -> END", file=sys.stderr)
219         return "end"
220
221     proposal = state.get("planner_proposal")
222     if not proposal:
223         print(" [Router] no proposal yet -> planner", file=sys.stderr)
224         return "planner"
225
226     feedback = state.get("reviewer_feedback")
227     if not feedback:
228         print(" [Router] proposal exists, unreviewed -> reviewer", file=sys.stderr)
229         return "reviewer"
230
231     if feedback.get("valid"):
232         print(" [Router] proposal valid -> END", file=sys.stderr)
233         return "end"
234
235     print(" [Router] proposal invalid -> planner (retry)", file=sys.stderr)
236     return "planner"
237
238

```

```

238
239 # -----
240 # Step 5: Assembling the graph
241 # -----
242 def build_graph():
243     graph = StateGraph(AgentState)
244     graph.add_node("supervisor", supervisor_node)
245     graph.add_node("planner", planner_node)
246     graph.add_node("reviewer", reviewer_node)
247
248     graph.set_entry_point("supervisor")
249
250     graph.add_conditional_edges(
251         "supervisor",
252         router_logic,
253         {"planner": "planner", "reviewer": "reviewer", "end": END},
254     )
255     graph.add_edge("planner", "supervisor")
256     graph.add_edge("reviewer", "supervisor")
257
258     return graph.compile()
259
260
261 # -----
262 # Step 6: Running and testing
263 # -----
264 def build_initial_state(
265     title: str,
266     content: str,
267     email: str = "",
268     strict: bool = True,
269     task: str = DEFAULT_TASK,
270     llm: Optional[Any] = None,
271     turn_ceiling: int = DEFAULT_TURN_CEILING,
272 ) -> AgentState:
273     return {
274         "title": title,
275         "content": content,
276         "email": email,
277         "strict": strict,
278         "task": task,
279         "llm": llm,
280         "planner_proposal": {},
281         "reviewer_feedback": {},
282         "turn_count": 0,
283         "turn_ceiling": turn_ceiling,
284     }
285
286
287 def run_pipeline(
288     title: str,
289     content: str,

```

```

289     content: str,
290     email: str = "",
291     strict: bool = True,
292     task: str = DEFAULT_TASK,
293     llm: Optional[Any] = None,
294     turn_ceiling: int = DEFAULT_TURN_CEILING,
295     stream: bool = True,
296 ) -> dict:
297     app = build_graph()
298     initial_state = build_initial_state(
299         title, content, email=email, strict=strict, task=task,
300         llm=llm, turn_ceiling=turn_ceiling,
301     )
302
303     final_state = dict(initial_state)
304     if stream:
305         for step in app.stream(initial_state):
306             for node_name, update in step.items():
307                 print(f"[stream] node={node_name} update={json.dumps(update, default=str)}", file=sys.stderr)
308                 final_state.update(update)
309     else:
310         final_state = app.invoke(initial_state)
311
312     feedback = final_state.get("reviewer_feedback") or {}
313     if feedback.get("valid"):
314         return {"tags": feedback["tags"], "summary": feedback["summary"], "abandoned": False}
315
316     return {
317         "tags": feedback.get("tags", []),
318         "summary": feedback.get("summary", ""),
319         "abandoned": True,
320         "error": feedback.get("error", "turn ceiling reached before a valid result"),
321     }
322
323
324 if __name__ == "__main__":
325     parser = argparse.ArgumentParser(
326         description="Run the Supervisor/Planner/Reviewer transit-incident tagging graph."
327     )
328     parser.add_argument("--turn-ceiling", type=int, default=DEFAULT_TURN_CEILING)
329     parser.add_argument("--strict", action="store_true", default=True)
330     parser.add_argument("--no-strict", dest="strict", action="store_false")
331     parser.add_argument("--llm", type=str, default=None, help="Optional model override, e.g. qwen3:8b")
332     args = parser.parse_args()
333
334     sample_title = "Route 22 -- Mechanical Breakdown"
335     sample_content = (
336         "incident_type=Mechanical Breakdown; transit_mode=Bus; "
337         "route_or_line=Route 22; location=Downtown Transit Center; "
338         "severity=Moderate; delay_minutes=20; description=A Route 22 bus "
339         "experienced a mechanical breakdown near the Downtown Transit "

```

```
338     "severity=Moderate; delay_minutes=20; description=A Route 22 bus "  
339     "experienced a mechanical breakdown near the Downtown Transit "  
340     "Center during evening rush hour. The engine stalled at a signal "  
341     "and could not restart, blocking one lane of traffic. Riders were "  
342     "transferred to the following bus after a 20-minute delay. A "  
343     "maintenance crew towed the disabled vehicle off-route roughly 45 "  
344     "minutes after the initial report."  
345 )  
346  
347 ✓ result = run_pipeline(  
348     sample_title, sample_content,  
349     strict=args.strict, llm=args.llm, turn_ceiling=args.turn_ceiling,  
350 )  
351 print(json.dumps(result))  
352
```

src >  model_client.py > ...

```
1  import json
2  import sys
3  import requests
4  from typing import Any, Dict, List, Optional
5
6  OLLAMA_URL = "http://localhost:11434/api/chat"
7  DEFAULT_MODEL = "qwen3:8b"
8  MAX_RETRIES = 3
9  DEFAULT_TIMEOUT = 240
10
11
12  class ModelClient:
13      def __init__(
14          self,
15          model: str = DEFAULT_MODEL,
16          base_url: str = OLLAMA_URL,
17          timeout: int = DEFAULT_TIMEOUT,
18          max_retries: int = MAX_RETRIES,
19      ):
20          self.model = model
21          self.base_url = base_url
22          self.timeout = timeout
23          self.max_retries = max_retries
24
25      def complete(
26          self,
27          messages: List[Dict[str, str]],
28          tools: Optional[List[Dict[str, Any]]] = None,
29          temperature: float = 0.0,
30          response_format: Optional[str] = "json",
31      ) -> Dict[str, Any]:
32          payload: Dict[str, Any] = {
33              "model": self.model,
34              "messages": messages,
35              "stream": False,
36              "think": False,
37              "options": {"temperature": temperature},
38          }
39
40          if response_format:
41              payload["format"] = response_format
42
43          if tools:
44              payload["tools"] = tools
45
46          last_error = None
47          for attempt in range(1, self.max_retries + 1):
48              try:
49                  resp = requests.post(
50                      self.base_url, json=payload, timeout=self.timeout
51                  )
52                  resp.raise_for_status()
53                  data = resp.json()
```

```

12 class ModelClient:
13     def complete(
14         messages: List[Dict[str, str]],
15         tools: Optional[List[Dict[str, Any]]] = None,
16         temperature: float = 0.0,
17         response_format: Optional[str] = "json",
18     ) -> Dict[str, Any]:
19         _default_client.complete(
20             messages=messages,
21             tools=tools,
22             temperature=temperature,
23         )
24
25     def complete(
26         self,
27         messages: List[Dict[str, str]],
28         tools: Optional[List[Dict[str, Any]]] = None,
29         temperature: float = 0.0,
30         response_format: Optional[str] = "json",
31     ) -> Dict[str, Any]:
32         resp = self._client.post(
33             url=self._url,
34             data={
35                 "messages": messages,
36                 "tools": tools,
37                 "temperature": temperature,
38                 "stream": False,
39                 "format": response_format,
40             },
41             headers={"Content-Type": "application/json"},
42         )
43
44         resp.raise_for_status()
45         data = resp.json()
46
47         message = data.get("message", {})
48
49         # Extract token usage from Ollama's standard response fields
50         input_tokens = data.get("prompt_eval_count", 0)
51         output_tokens = data.get("eval_count", 0)
52
53         if "tool_calls" in message:
54             res = {"tool_calls": message["tool_calls"]}
55         else:
56             content = message.get("content", "")
57             if response_format == "json":
58                 res = json.loads(content)
59             else:
60                 res = {"content": content}
61
62         res["_usage"] = {
63             "input_tokens": input_tokens,
64             "output_tokens": output_tokens,
65             "total_tokens": input_tokens + output_tokens,
66         }
67
68         return res
69
70     except (requests.RequestException, KeyError, json.JSONDecodeError) as exc:
71         last_error = exc
72         print(
73             f" [retry {attempt}/{self.max_retries}] model call failed: {exc}",
74             file=sys.stderr,
75         )
76
77         raise RuntimeError(
78             f"Model call failed after {self.max_retries} attempts: {last_error}"
79         )
80
81 _default_client = ModelClient()
82
83 def complete(
84     messages: List[Dict[str, str]],
85     tools: Optional[List[Dict[str, Any]]] = None,
86     temperature: float = 0.0,
87     response_format: Optional[str] = "json",
88 ) -> Dict[str, Any]:
89     return _default_client.complete(
90         messages=messages,
91         tools=tools,
92         temperature=temperature,
93     )

```

```

(base) Nathans-MBP:code nathanpak$ FORCE_INVALID=1 python agent_graph.py
---NODE: Supervisor (turn 1) ---
[Router] no proposal yet -> planner
[stream] node=supervisor update={"turn_count": 1}
---NODE: Planner ---
[stream] node=planner update={"planner_proposal": {"candidate_tags": ["mechanical breakdown", "bus route 22",
, "downtown transit center", "evening rush hour", "moderate severity", "traffic lane blocked", "maintenance
crew response"], "draft_summary": "A Route 22 bus experienced a mechanical breakdown near the Downtown Trans
it Center during evening rush hour, causing a moderate delay and blocking one lane of traffic. Riders were t
ransferred to the following bus after a 20-minute delay, with the disabled vehicle towed off-route approxima
tely 45 minutes later."}, "reviewer_feedback": {}}}
---NODE: Supervisor (turn 2) ---
[Router] proposal exists, unreviewed -> reviewer
[stream] node=supervisor update={"turn_count": 2}
---NODE: Reviewer ---
[Reviewer] FORCE_INVALID=1 set -> reporting a fake issue
[stream] node=reviewer update={"reviewer_feedback": {"valid": false, "error": "FORCE_INVALID debug mode: pre
tending validation failed."}}
---NODE: Supervisor (turn 3) ---
[Router] proposal invalid -> planner (retry)
[stream] node=supervisor update={"turn_count": 3}
---NODE: Planner ---
[stream] node=planner update={"planner_proposal": {"candidate_tags": ["mechanical breakdown", "bus route 22",
, "downtown transit center", "evening rush hour", "moderate severity"], "draft_summary": "Route 22 bus had m
echanical breakdown at downtown transit center during evening rush hour, causing 20-minute delay and lane bl
ockage."}, "reviewer_feedback": {}}}
---NODE: Supervisor (turn 4) ---
[Router] proposal exists, unreviewed -> reviewer
[stream] node=supervisor update={"turn_count": 4}
---NODE: Reviewer ---
[Reviewer] FORCE_INVALID=1 set -> reporting a fake issue
[stream] node=reviewer update={"reviewer_feedback": {"valid": false, "error": "FORCE_INVALID debug mode: pre
tending validation failed."}}
---NODE: Supervisor (turn 5) ---
[Router] proposal invalid -> planner (retry)
[stream] node=supervisor update={"turn_count": 5}
---NODE: Planner ---
[stream] node=planner update={"planner_proposal": {"candidate_tags": ["mechanical breakdown", "bus route 22",
, "downtown transit center", "evening rush hour", "moderate severity"], "draft_summary": "Route 22 bus had m
echanical breakdown at downtown transit center during evening rush hour, causing 20-minute delay and lane bl
ockage."}, "reviewer_feedback": {}}}
---NODE: Supervisor (turn 6) ---
[Router] turn ceiling (6) reached -> END
[stream] node=supervisor update={"turn_count": 6}
{"tags": [], "summary": "", "abandoned": true, "error": "turn ceiling reached before a valid result"}
(base) Nathans-MBP:code nathanpak$

```

Part 4)

```

24 import argparse
25 import csv
26 import json
27 import os
28 import statistics
29 import sys
30 import time
31 from datetime import datetime, timezone
32
33 import agent_graph as ag
34
35 REPORTS_DIR = "reports/hw02"
36 CASES_DIR = os.path.join(REPORTS_DIR, "cases")
37 RAW_DIR = os.path.join(REPORTS_DIR, "raw")
38
39 SCHEMA_INPUT_PATH = os.path.join(CASES_DIR, "schema_input.json")
40 ADVERSARIAL_INPUT_PATH = os.path.join(CASES_DIR, "adversarial_input.json")
41
42 # --- Part 4 Sec 3/4: fixed domain input, frozen before the experiment ---
43 FROZEN_TITLE = "Route 22 -- Mechanical Breakdown"
44 FROZEN_CONTENT = (
45     "incident_type=Mechanical Breakdown; transit_mode=Bus; "
46     "route_or_line=Route 22; location=Downtown Transit Center; "
47     "severity=Moderate; delay_minutes=20; description=A Route 22 bus "
48     "experienced a mechanical breakdown near the Downtown Transit "
49     "Center during evening rush hour. The engine stalled at a signal "
50     "and could not restart, blocking one lane of traffic. Riders were "
51     "transferred to the following bus after a 20-minute delay. A "
52     "maintenance crew towed the disabled vehicle off-route roughly 45 "
53     "minutes after the initial report."
54 )
55
56 # --- Part 4 Sec 5: adversarial input ---
57 # Deliberately sparse and repetitive: almost no concrete, distinct details
58 # to ground 3 *distinct* tags in (everything is "unknown" or hedged), which
59 # tends to push the Planner toward vague/overlapping candidate tags and the
60 # Reviewer toward either duplicate-ish picks or a rambling, hedge-filled
61 # summary that creeps past the 25-word cap.

```

```

60 # Reviewer toward either duplicate-ish picks or a rambling, hedge-filled
61 # summary that creeps past the 25-word cap.
62 ADVERSARIAL_TITLE = "Incident"
63 ADVERSARIAL_CONTENT = (
64     "incident_type=Other; transit_mode=Bus; route_or_line=Unknown; "
65     "location=Unknown; severity=Low; description=Something happened, "
66     "possibly, though it is not fully clear what. It may or may not have "
67     "affected service in some way, possibly related to a bus, or maybe "
68     "not, riders were unsure, staff were unsure, no further details were "
69     "recorded, and it is not clear if this was even a real incident or "
70     "a false report."
71 )
72
73
74 def ensure_dirs():
75     os.makedirs(CASES_DIR, exist_ok=True)
76     os.makedirs(RAW_DIR, exist_ok=True)
77
78
79 def save_case(path, title, content):
80     if os.path.exists(path):
81         return # frozen -- don't overwrite an input already used for a run
82     with open(path, "w") as f:
83         json.dump({"title": title, "content": content}, f, indent=2)
84
85
86 def load_case(path):
87     with open(path) as f:
88         data = json.load(f)
89     return data["title"], data["content"]
90
91
92 def log(msg):
93     ts = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
94     line = f"[{ts}] {msg}"
95     print(line)
96     with open(os.path.join(REPORTS_DIR, "RUN_LOG.txt"), "a") as f:
97         f.write(line + "\n")
98
99
100 def classify(result: dict) -> str:
101     if result.get("abandoned"):
102         return "hit_turn_ceiling"
103     attempts = result.get("attempts", 1)

```

```

101     if result.get("abandoned"):
102         return "hit_turn_ceiling"
103     attempts = result.get("attempts", 1)
104     if attempts <= 1:
105         return "valid_first_attempt"
106     if attempts == 2:
107         return "valid_after_1_retry"
108     return "valid_after_2plus_retries"
109
110     (function) def perf_counter() -> float
111     def run_single(title, content, strict=strict, turn_ceiling=turn_ceiling, stream=False,
112                   start = time.perf_counter(),
113                   result = ag.run_pipeline(
114                       title, content, strict=strict, turn_ceiling=turn_ceiling, stream=False,
115                   )
116                   latency_ms = (time.perf_counter() - start) * 1000
117                   result["latency_ms"] = round(latency_ms, 1)
118                   result["classification"] = classify(result)
119                   return result
120
121
122     def write_raw(records, path_json, path_csv):
123         with open(path_json, "w") as f:
124             json.dump(records, f, indent=2)
125         if records:
126             keys = sorted({k for r in records for k in r.keys()})
127             with open(path_csv, "w", newline="") as f:
128                 writer = csv.DictWriter(f, fieldnames=keys)
129                 writer.writeheader()
130                 for r in records:
131                     row = {k: r.get(k, "") for k in keys}
132                     for k, v in row.items():
133                         if isinstance(v, (list, dict)):
134                             row[k] = json.dumps(v)
135                 writer.writerow(row)
136
137
138     def summarize(records):
139         buckets = [
140             "valid_first_attempt", "valid_after_1_retry",
141             "valid_after_2plus_retries", "hit_turn_ceiling",
142         ]

```

```

141     "valid_after_2plus_retries", "hit_turn_ceiling",
142 ]
143 summary = {}
144 for b in buckets:
145     matching = [r["latency_ms"] for r in records if r["classification"] == b]
146     summary[b] = {
147         "count": len(matching),
148         "mean_latency_ms": round(statistics.mean(matching), 1) if matching else None,
149     }
150 return summary
151
152
153 def print_metrics_table(header, summary):
154     label_map = {
155         "valid_first_attempt": "Valid first attempt",
156         "valid_after_1_retry": "Valid after 1 retry",
157         "valid_after_2plus_retries": "Valid after 2+ retries",
158         "hit_turn_ceiling": "Hit turn ceiling",
159     }
160     print(f"\n{header}")
161     print(f"{'Outcome':<28}{'Count':<8}{'Mean latency (ms)':<20}")
162     for key, label in label_map.items():
163         s = summary[key]
164         mean = s["mean_latency_ms"] if s["mean_latency_ms"] is not None else "-"
165         print(f"{label:<28}{s['count']:<8}{mean:<20}")
166
167
168 # -----
169 # Part 4 Sec 3: 30 runs on the frozen schema input, classified
170 # -----
171 def experiment_schema_validation(n=30, turn_ceiling=ag.DEFAULT_TURN_CEILING):
172     ensure_dirs()
173     save_case(SCHEMA_INPUT_PATH, FROZEN_TITLE, FROZEN_CONTENT)
174     title, content = load_case(SCHEMA_INPUT_PATH)
175
176     log(f"=== Part 4 Sec 3: schema-validation experiment, n={n}, turn_ceiling={turn_ceiling} ===")
177     records = []
178     for i in range(1, n + 1):
179         result = run_single(title, content, turn_ceiling)
180         result["run_index"] = i
181         records.append(result)
182     log(

```

```

182         log(
183             f"run {i}/{n}: classification={result['classification']} "
184             f"attempts={result.get('attempts')} latency_ms={result['latency_ms']}"
185         )
186
187     write_raw(
188         records,
189         os.path.join(RAW_DIR, "schema_validation_runs.json"),
190         os.path.join(RAW_DIR, "schema_validation_runs.csv"),
191     )
192
193     summary = summarize(records)
194     log(f"schema-validation summary: {json.dumps(summary)}")
195     print_metrics_table("Part 4 Sec 3 -- Outcome over 30 runs", summary)
196     return records, summary
197
198
199 # -----
200 # Part 4 Sec 4: ceiling comparison, 2 vs 10, 20 runs each
201 # -----
202 def experiment_ceiling_comparison(n=20, ceilings=(2, 10)):
203     ensure_dirs()
204     save_case(SCHEMA_INPUT_PATH, FROZEN_TITLE, FROZEN_CONTENT)
205     title, content = load_case(SCHEMA_INPUT_PATH) # same frozen input as Sec 3
206
207     log(f"=== Part 4 Sec 4: turn-ceiling comparison, n={n} per ceiling, ceilings={ceilings} ===")
208     all_records = []
209     results_by_ceiling = {}
210     for ceiling in ceilings:
211         records = []
212         for i in range(1, n + 1):
213             result = run_single(title, content, ceiling)
214             result["run_index"] = i
215             result["turn_ceiling"] = ceiling
216             records.append(result)
217             all_records.append(result)
218             log(
219                 f"ceiling={ceiling} run {i}/{n}: valid={not result['abandoned']} "
220                 f"latency_ms={result['latency_ms']}"
221             )
222
223     completed = [r for r in records if not r["abandoned"]]
224     completion_rate = len(completed) / len(records)

```

```

223     completed = [r for r in records if not r["abandoned"]]
224     completion_rate = len(completed) / len(records)
225     mean_latency = statistics.mean(r["latency_ms"] for r in records)
226     results_by_ceiling[ceiling] = {
227         "completion_rate": round(completion_rate, 3),
228         "mean_latency_ms": round(mean_latency, 1),
229     }
230     log(
231         f"ceiling={ceiling} summary: completion_rate={completion_rate:.1%} "
232         f"mean_latency_ms={mean_latency:.1f}"
233     )
234
235     write_raw(
236         all_records,
237         os.path.join(RAW_DIR, "ceiling_comparison_runs.json"),
238         os.path.join(RAW_DIR, "ceiling_comparison_runs.csv"),
239     )
240
241     print(f"\nPart 4 Sec 4 -- Turn ceiling comparison (n={n} each)")
242     print(f"{'Ceiling':<10}{'Completion rate':<18}{'Mean latency (ms)':<20}")
243     for ceiling, s in results_by_ceiling.items():
244         print(f"{'ceiling':<10}{s['completion_rate']:<18}{s['mean_latency_ms']:<20}")
245
246     faster = min(results_by_ceiling, key=lambda c: results_by_ceiling[c]["mean_latency_ms"])
247     more_complete = max(results_by_ceiling, key=lambda c: results_by_ceiling[c]["completion_rate"])
248     print(
249         f"\nFaster ceiling: {faster} "
250         f"({results_by_ceiling[faster]['mean_latency_ms']} ms mean). "
251         f"Higher completion rate: {more_complete} "
252         f"({results_by_ceiling[more_complete]['completion_rate']:.1%})."
253     )
254     print(
255         "Use this pair of numbers to justify your deployment pick in "
256         "METRICS.md -- e.g. 'ceiling=10 chosen: completion rate rose from "
257         "X% to Y% for only Z ms extra mean latency', or the reverse if the "
258         "lower ceiling barely loses completion rate for much less latency."
259     )
260     return results_by_ceiling
261
262
263 # -----
264 # Part 4 Sec 5: adversarial input, 5 runs
265 # -----

```

```

265 # -----
266 ✓ def experiment_adversarial(n=5, turn_ceiling=ag.DEFAULT_TURN_CEILING):
267     ensure_dirs()
268     save_case(ADVERSARIAL_INPUT_PATH, ADVERSARIAL_TITLE, ADVERSARIAL_CONTENT)
269     title, content = load_case(ADVERSARIAL_INPUT_PATH)
270
271     log(f"=== Part 4 Sec 5: adversarial input, n={n}, turn_ceiling={turn_ceiling} ===")
272     records = []
273     for i in range(1, n + 1):
274         result = run_single(title, content, turn_ceiling)
275         result["run_index"] = i
276         records.append(result)
277         log(
278             f"adversarial run {i}/{n}: abandoned={result['abandoned']} "
279             f"attempts={result.get('attempts')} latency_ms={result['latency_ms']}"
280         )
281
282     write_raw(
283         records,
284         os.path.join(RAW_DIR, "adversarial_runs.json"),
285         os.path.join(RAW_DIR, "adversarial_runs.csv"),
286     )
287
288     hit_ceiling = sum(1 for r in records if r["abandoned"])
289     rate = hit_ceiling / len(records)
290     log(f"adversarial summary: hit_ceiling={hit_ceiling}/{len(records)} ({rate:.1%}")
291     print(
292         f"\nPart 4 Sec 5 -- Adversarial input hit the turn ceiling in "
293         f"{hit_ceiling}/{len(records)} runs ({rate:.1%})."
294     )
295     if hit_ceiling < 4:
296         print(
297             "Note: this did not reach the ceiling in >=4/5 runs as preferred "
298             "by the assignment -- report the observed rate above rather than "
299             "claiming it's deterministic, and consider strengthening the "
300             "adversarial input (e.g. even sparser detail, or push the draft "
301             "summary closer to/over the 25-word limit) if you want a higher "
302             "failure rate."
303         )
304     return records, rate
305
306
307 ✓ if __name__ == "__main__":
308     parser = argparse.ArgumentParser(description="Part 4 experiment runner for HW2")

```

```

306
307     if __name__ == "__main__":
308         parser = argparse.ArgumentParser(description="Part 4 experiment runner for HW2.")
309         parser.add_argument(
310             "experiment", choices=["schema", "ceiling", "adversarial", "all"],
311         )
312         args = parser.parse_args()
313
314         ensure_dirs()
315         if args.experiment in ("schema", "all"):
316             experiment_schema_validation()
317         if args.experiment in ("ceiling", "all"):
318             experiment_ceiling_comparison()
319         if args.experiment in ("adversarial", "all"):
320             experiment_adversarial()
321

```

```

[(base) Nathans-MBP:code nathanpak$ python run_experiments.py all
[2026-09-14T19:41:42Z] === Part 4 Sec 3: schema-validation experiment, n=30, turn_ceiling=6 ===
---NODE: Supervisor (turn 1) ---
[Router] no proposal yet -> planner
---NODE: Planner ---
---NODE: Supervisor (turn 2) ---
[Router] proposal exists, unreviewed -> reviewer
---NODE: Reviewer ---
---NODE: Supervisor (turn 3) ---
[Router] proposal valid -> END
[2026-09-14T19:42:13Z] run 1/30: classification=valid_first_attempt attempts=1 latency_ms=30587.7
---NODE: Supervisor (turn 1) ---
[Router] no proposal yet -> planner
---NODE: Planner ---
---NODE: Supervisor (turn 2) ---
[Router] proposal exists, unreviewed -> reviewer
---NODE: Reviewer ---
---NODE: Supervisor (turn 3) ---
[Router] proposal valid -> END
[2026-09-14T19:42:31Z] run 2/30: classification=valid_first_attempt attempts=1 latency_ms=18065.2
---NODE: Supervisor (turn 1) ---
[Router] no proposal yet -> planner
---NODE: Planner ---
---NODE: Supervisor (turn 2) ---
[Router] proposal exists, unreviewed -> reviewer
---NODE: Reviewer ---
---NODE: Supervisor (turn 3) ---
[Router] proposal valid -> END
[2026-09-14T19:42:49Z] run 3/30: classification=valid_first_attempt attempts=1 latency_ms=18371.9
---NODE: Supervisor (turn 1) ---
[Router] no proposal yet -> planner
---NODE: Planner ---
---NODE: Supervisor (turn 2) ---
[Router] proposal exists, unreviewed -> reviewer
---NODE: Reviewer ---
---NODE: Supervisor (turn 3) ---
[Router] proposal valid -> END
[2026-09-14T19:43:08Z] run 4/30: classification=valid_first_attempt attempts=1 latency_ms=18666.8
---NODE: Supervisor (turn 1) ---
[Router] no proposal yet -> planner
---NODE: Planner ---
---NODE: Supervisor (turn 2) ---
[Router] proposal exists, unreviewed -> reviewer
---NODE: Reviewer ---
---NODE: Supervisor (turn 3) ---

```